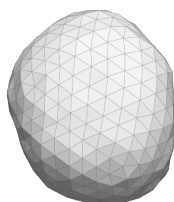


Numen

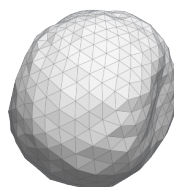
A Hybrid Proof-of-Work + BFT Layer 1 with Proof of Scan Mining

Every block is real work you can look at: miners sculpt and scan a deterministic 3D asteroid, and GRANDPA makes the result irreversible in about a minute.

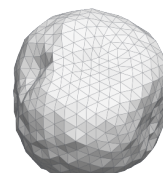
seed 0x2A... → asteroid #101



seed 0x9C... → asteroid #102



seed 0xF3... → asteroid #103



Deterministic asteroids. Every sealed block reconstructs a unique on-chain artifact from its nonce.

Abstract. Numen is a public Layer-1 blockchain built on the Polkadot SDK (Substrate) that pairs permissionless Proof-of-Work block production with the Byzantine-fault-tolerant finality gadget GRANDPA, so that anyone can mine anonymously while transactions become irreversible within about a minute. Its work function is Proof of Scan (PoScan). PoScan trades opaque double-hashing for verifiable geometric labour. Every nonce derives a full-width seed from the protocol version and `pre_hash`, which itself commits to the parent block and the block author. The miner generates a 2,562-vertex asteroid mesh from that seed, scans it through the `spectra13d` pipeline into a 23-dimensional shape spectrum, and quantizes the spectrum on a mining-grade fine grid. Hashing the buckets with `pre_hash` gives a full-entropy work value. The scan itself is the work: low-resolution, surrogate, and precomputation shortcuts are closed by measurement or by construction, and the winning nonce doubles as a reconstructible on-chain artifact. One block, one asteroid. Numen drops recognition semantics on purpose. It keeps the scan and refuses the object-identity claim, because that claim cannot survive in a consensus path. Validators are identity-bearing and coin-locked, with no slashing and, deliberately, no reward; an on-chain exemption list makes entry invitation-based in the early network, and the stake threshold is priced so that buying finality is arithmetically impossible. The chain ships full EVM compatibility through Frontier. Governance removes sudo entirely: a project key (`prime`) can put code to a vote, pull the brakes, and keep two administrative gates, but it can never move funds, and all of this is verifiable on-chain. We document the open limitations plainly: v1 does not target FPGA/ASIC resistance, and no third-party audit has yet been performed.

Contents

1	Introduction and vision	5
1.1	Motivation: work you can look at	5
1.2	The name, and what it promises	5
1.3	Why recognition cannot be the work	5
1.4	The reframing: the scan is the work, not the identity	6
1.5	What Numen claims, and refuses to claim	6
2	Design principles	7
3	System architecture	7
4	Consensus: PoW production, BFT finality	8
4.1	Block production	8
4.2	Fork choice	9
4.3	GRANDPA finality and sessions	9
4.4	Two-layer security model	9
5	Proof of Scan: the mining algorithm	9
5.1	Seed derivation: binding the work	10
5.2	Deterministic asteroid generation	11
5.3	Spectral scan: the work itself	11
5.4	Fine quantization: restoring full entropy	11
5.5	Seal and verification	11
5.6	Why shortcuts fail	12
5.7	Bit-level determinism	12
5.8	One block, one asteroid	13
6	Difficulty regulation: ASERT with outage re-anchoring	14
6.1	Absolutely scheduled exponential targets	14
6.2	Recovery from hash-power collapse	14
7	GRANDPA and validators	15
7.1	GRANDPA's fragility	15
7.2	Why entry is by invitation	15
7.3	Is the invitation era another monopoly?	15
7.4	Who gets invited	16
7.5	Two keys, never mixed	16
7.6	Liveness enforcement	16

7.7	No slashing and no reward, by design	17
7.8	Lock mechanics	17
8	Mining service: built-in and external	18
8.1	Reconstructing any block's asteroid	18
9	EVM compatibility	19
9.1	Native-token ERC20 precompile	19
10	Account features	19
11	Tokenomics and monetary policy	20
11.1	Genesis issuance: no team pre-mine	21
11.2	Mining issuance: a halving curve to a fixed ceiling	21
11.3	Fees, and where value accrues	21
12	Governance and treasury	22
12.1	OpenGov and the treasury	22
12.2	Treasury culture: deliver, then pay	23
13	Sudo and prime	23
13.1	Prime's privileges	24
13.2	Why these privileges sit with prime	24
13.3	The path to decentralization is left open	25
13.4	Who holds prime	25
14	Security analysis	25
14.1	Propagation and the orphan budget	25
14.2	Attack surface	26
14.3	Known risks	27
15	Risk factors	28
16	Disclaimer	28
17	Glossary	28
	Appendix A: Governance key set and protocol string	30
	Core contributors	30

On form, number, and the nature of things

“All nature then, as it is of itself, is built of these two things: for there are bodies and the void, in which they are placed and where they move hither and thither.”

— Lucretius, *De Rerum Natura* I.419–421 (c. 55 BCE), trans. Cyril Bailey

Numen resolves every asteroid it scans into discrete bodies. What comes out is a finite vector of quantized feature buckets. Matter, and the empty grid between its values.

“Philosophy is written in this grand book—I mean the Universe—which stands continually open to our gaze, but it cannot be understood unless one first learns to comprehend the language and interpret the characters in which it is written. It is written in the language of mathematics, and its characters are triangles, circles, and other geometrical figures...”

— Galileo Galilei, *The Assayer* (1623), trans. Stillman Drake

Numen's proof-of-work reads a book written in triangles. A 2,562-vertex mesh, measured into number. Geometry is the labour.

“The body is the soul's material basis; the 'soul' is the functioning of the body.”

— Fan Zhen, *Essay on the Extinction of the Soul* (c. 507 CE), trans. Etienne Balazs

The shape is the material basis; the hash is its functioning. The hash manifests the shape, and no separate soul could be forged apart from it.

1 Introduction and vision

1.1 Motivation: work you can look at

Proof-of-Work remains the only battle-tested way to let strangers mint blocks without permission, identity, or stake. Yet the work itself is a lottery of opaque hashes: society sees the energy bill but never the labour. The field has a recurring ambition called *useful PoW*. It asks whether the computation can leave something behind. Attempts to make mining useful to the outside world (protein folding, machine-learning training) consistently stall on three structural problems: decentralized task supply, verification as expensive as the work, and the double-selling of results off-chain. The one direction that survives scrutiny is *internal* usefulness: let the work itself produce an artifact the chain can own.

Numen adopts that honest, narrow definition. Every block is the trace of real geometric labour, the deterministic sculpting and spectral scanning of a small asteroid, and the winning nonce reconstructs that asteroid forever. At the same time, pure Nakamoto consensus leaves users with only probabilistic finality; exchanges wait dozens of confirmations. Numen therefore pairs open PoW block production with GRANDPA [2], the Substrate ecosystem's BFT finality protocol. An independent, coin-locked validator set operates it. Miner and validator are separate roles that never mix.

1.2 The name, and what it promises

Numen is the Latin word for the divinity the Romans believed inhabited each concrete thing. It is the animating presence of *this* particular spring, stone, or threshold. The name carries two deliberate overtones. *Noumenon* is Kant's thing-in-itself. *Númenor* is the island of the far-sighted. It was chosen because it maps precisely onto the technical core. Numen turns the three-dimensional shape of an object into a stable digital fingerprint; it gives each concrete thing its own presence on-chain. Fan Zhen's thesis is the design in one line: the form is the substance, and the fingerprint is merely the form's function; there is no separate soul to be forged apart from the shape.

That promise comes with a discipline the rest of this paper keeps carefully. The mapping "shape → fingerprint" is real and load-bearing *inside* consensus; it is not a claim that Numen authenticates physical objects in the world. Section 1.3 explains why that stronger claim is a dead end, and Section 1.5 states exactly where the line falls.

1.3 Why recognition cannot be the work

There is a tempting way to put 3D scanning into a blockchain: let miners process real 3D objects, treat an object's recognition hash as its durable identity, and sell the result as "1 object = 1 asset." Numen studied this design space closely before rejecting it, because every property that makes a recognition pipeline good at recognizing makes it wrong for consensus.

First, the mechanism problems that make recognition-as-consensus unsafe:

- **Perceptual hashing is anti-avalanche by design.** A recognition hash must map similar shapes to the *same* value; mining needs the opposite. With identity-grade quantization the reachable work space collapses to a few thousand values. A laptop enumerates it in minutes, difficulty loses meaning, and mining degenerates into a lookup table.
- **Work not bound to the block or the miner.** An object's hash has no parent hash inside it: it can be precomputed, replayed across blocks, or stolen from another miner's submission.

- **Miner-supplied objects invite degenerate and forged meshes.** A 12-vertex sliver scans almost for free (work collapses), and a forge-collide experiment shows a 12-vertex mesh can impersonate the coarse-bucket identity of a 2,562-vertex object, producing counterfeit “winning shapes.”
- **Floating-point features sit in consensus.** Recognition pipelines are $\mathbb{F}_{64}$ geometry; cross-platform math divergence at a bucket boundary is a consensus fork.
- **Shape-only identity is weaker than it looks.** Silhouette-level features miss internal geometry and collide across scaled or same-outline objects. The feature-entropy ceiling also makes the hash reversible to a coarse shape. That is bounded leakage, not zero-knowledge.

Second, and decisively, the product built on shape-only recognition does not survive contact with reality. Contour-level recognition loses internal structure and is scale-sensitive, so distinct objects with similar outlines collide, and a modified object with an unchanged silhouette still matches. Copyright registration, luxury-goods authentication, physical-object identity, manufacturing verification: each intended application dies on that same defect. An authenticity oracle that a same-silhouette replica can satisfy is not an oracle.

The honest conclusion is not that scanning has no place in consensus; it is that recognition output cannot be the entropy source, and object authenticity cannot be the product.

1.4 The reframing: the scan is the work, not the identity

PoScan as specified here inverts every failing assumption of Section 1.3.

- **The chain, not the miner, chooses the object.** The mesh comes from a seed the chain fixes, never from anything the miner supplies (Section 5.2). Trivial objects, forged objects, and replays all die at once, because verifiers regenerate the mesh and there is nothing to submit but a nonce.
- **Quantization is mining-grade, not identity-grade.** The 23 spectral features are quantized about 10^5 times finer than recognition buckets (Section 5.4). At that resolution the channel stops being enumerable, and the final SHA-256 turns it into a uniform 256-bit work value.
- **Identity claims are simply dropped.** The asteroid is an artifact, not an asset-authenticity oracle. Nothing in consensus asserts that two similar shapes are “the same thing”; the anti-avalanche property that poisoned mining is discarded rather than patched.

What survives from the scan-as-identity vision is exactly the part that was always sound: a scan is honest, replayable geometric labour. Making the miner perform it, and making every shortcut measurably fail, is the substance of Section 5.

1.5 What Numen claims, and refuses to claim

Numen is precise about its central sentence, because the difference between the true and the tempting version is the difference between a defensible protocol and a marketing claim an expert dismisses on sight.

We claim: block production requires executing the `spectral3d` scan of a chain-chosen mesh; the scan is the work; and any approximate or shortcut implementation computes a different value and forfeits the block. This rests on the bit-level determinism of a single reference implementation (Section 5.7).

We do not claim: that shape “invariance” or noise robustness secures mining. Once the seed fixes the mesh, its pose is fixed; invariance plays no role in mining and is deliberately idle. The load-bearing property is *avalanche*. Any deviation yields a wildly different work value. That is the exact opposite of the recognition property. A white paper that sold invariance as the security base would be wrong, and we say so plainly.

2 Design principles

- **PoW decides who speaks; BFT decides what is final.** Anyone may mine anonymously; irreversibility comes from an explicit vote of an accountable validator set.
- **Reorgs before finality are normal.** The protocol does not fight short PoW races; it bounds them with fast finality.
- **Miners and validators are disjoint roles.** No overlap is required or assumed; either side can fail without taking the other down.
- **Verification is cheap, deterministic, and runs in the runtime.** Seal verification executes inside the wasm runtime (`PowVerifyApi`), so the PoW algorithm itself can be replaced by a forkless runtime upgrade.
- **Determinism before cleverness.** Every consensus computation must reproduce bit-for-bit across x86-64, aarch64, and wasm32; anything that cannot is kept out of the consensus path.
- **EVM compatibility first.** Developers and users keep their Ethereum tooling unchanged.
- **The project can ship code, pull the brakes, and keep gates. It can never move funds.** No super-user account, no route to Root but a referendum, and every privilege on that short list checkable in the runtime metadata (Section 12).
- **Fair launch, honest non-goals.** The project holds no pre-mined balance and earns fees by mining like anyone else; v1 does not claim FPGA/ASIC resistance, external usefulness, or recognition-based asset identity.

3 System architecture

Numen is a Substrate solochain [1] organized in five layers (Figure 1). The consensus layer couples a PoW import pipeline with a GRANDPA voter; the runtime hosts the chain logic as FRAME pallets (balances, validator management, sessions, online heartbeats, ASERT difficulty, treasury and referenda governance, the `prime` upgrade pallet, on-chain identity, vesting, and the `PowVerifyApi` runtime API that re-executes PoScan seals); the EVM layer embeds Frontier for full Ethereum compatibility. A public network of a few thousand nodes is the design target.

Three node roles emerge from configuration, not protocol: *mining nodes* (built-in miner or mining RPC enabled), *validator nodes* (session keys registered, coins locked or exempt), and plain *full / RPC nodes*. All of them verify everything; none of them trusts any other.

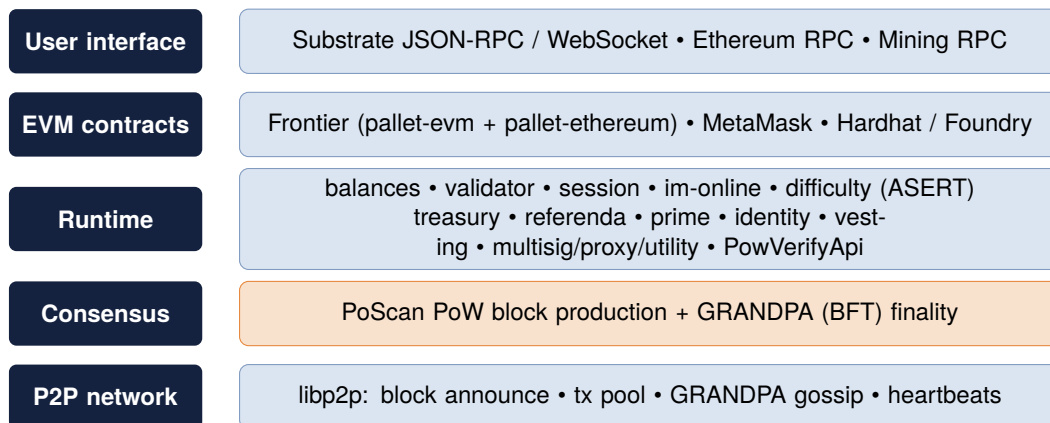


Figure 1 – five-layer architecture of a Numen node. The consensus layer (highlighted) is the only place where PoScan and GRANDPA meet.

4 Consensus: PoW production, BFT finality

Numen runs two consensus processes in parallel over one chain (Figure 2). Miners extend the chain by PoScan Proof-of-Work; validators finalize a growing prefix by GRANDPA. Block production is permissionless and probabilistic; finality is absolute and comes from a permissioned validator set.

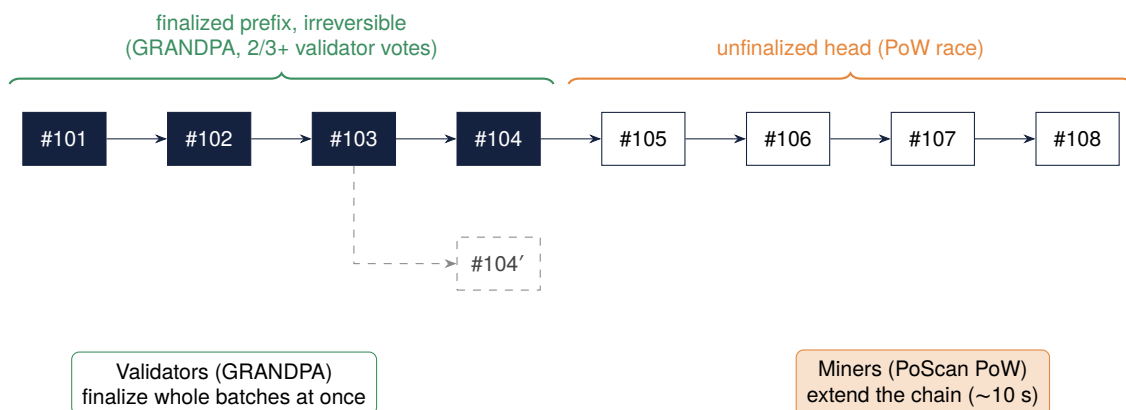


Figure 2 – the two lanes of consensus. Miners extend the head; validators finalize a growing prefix. A competing fork (#104') is allowed before finality and resolved by the heaviest-chain rule.

4.1 Block production

Blocks are produced competitively by PoScan mining at a target interval of 10 seconds, with a hard minimum spacing of 2 seconds, enforced on the timestamp rather than on arrival. Block construction is capped at 2 s, and import verification runs on a 5 s budget, keeping propagation-induced orphan rates within the budget of Section 14.1. The block author is the miner address configured on the producing node; it receives the block reward (initially 16 NUMN, halving; Section 11) plus its fee share and all tips. Orphaned and uncle blocks receive nothing: with 10-second blocks and fast finality, uncle accounting adds complexity without measurable fairness gain.

4.2 Fork choice

- **Heaviest chain.** The best chain is the one with the highest accumulated difficulty, not the greatest height.
- **Finality dominates weight.** Any branch conflicting with a GRANDPA-finalized block never becomes best, whatever its accumulated difficulty. This rule is enforced in the block-import pipeline and is the load-bearing joint of the hybrid design (Section 14.3).
- **First-seen tie-break.** Among equal-weight heads, a node keeps the chain it saw first, blunting selfish-mining release games.

4.3 GRANDPA finality and sessions

GRANDPA runs as an independent layer above PoW. Validators vote on *chains* rather than individual blocks; a round concluding with more than 2/3 of validator weight finalizes an entire prefix at once, so bursts of blocks are absorbed gracefully. With `gossip_duration = 2 s` a voting round completes within about 10 s, and a transaction goes from inclusion to irreversibility in about a minute. Finalized blocks are irreversible: no reorganization can cross them.

The validator set changes only at session boundaries (every 60 blocks, roughly 10 minutes). Joins and exits registered during a session take effect at the next boundary through GRANDPA's `ScheduledChange` mechanism, which lets the old set finish its last round before the handover (Section 14.3). One GRANDPA vote can finalize many blocks, so finality throughput is never the bottleneck.

4.4 Two-layer security model

Safety rests on two independent thresholds. The unfinalized frontier is protected by PoW: rewriting it requires a majority of hash power (>50%). Finalized history is protected by BFT: reverting it requires corrupting more than 1/3 of validator weight (Section 7.1). An attacker must beat both layers to rewrite settled history, and each layer alone degrades gracefully: losing GRANDPA (validators offline) leaves a functioning pure-PoW chain without finality; losing PoW majority still cannot touch finalized blocks. If the validator set empties out (everyone exited or kicked), the chain degrades to pure PoW until new validators lock in and GRANDPA resumes at the next session boundary.

5 Proof of Scan: the mining algorithm

PoScan replaces the inner hash function of classic PoW and nothing else. The seal format, the difficulty comparison, and the runtime verification framework are inherited unchanged from the standard SHA-256 PoW pipeline; what changes is how the 256-bit work value `s` is computed from `pre_hash` and the nonce. Figure 3 shows the five stages; each is specified below.

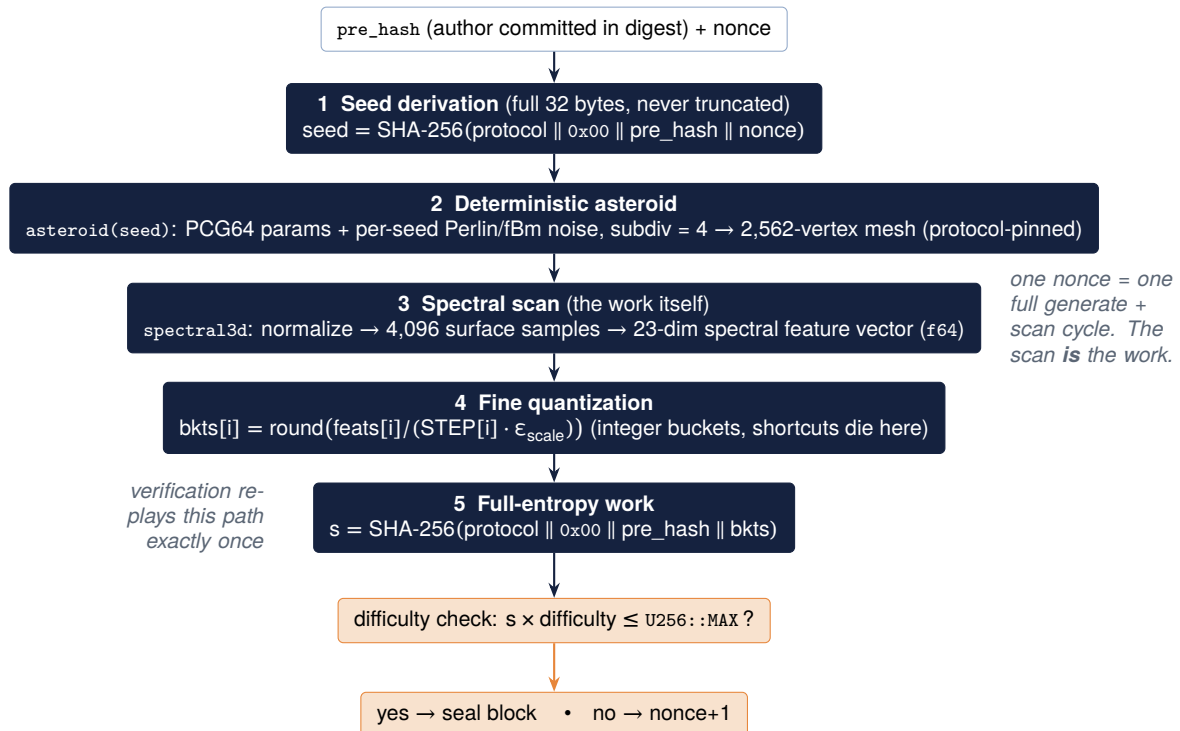


Figure 3 – the PoScan pipeline. Difficulty presses on a full-entropy SHA-256 value whose preimage can only be obtained by scanning a freshly generated mesh. The miner address is committed in the block’s pre-runtime digest, so it flows into `pre_hash` and binds the work; `pre_hash` also enters the final hash directly, so no precomputed table survives a block change.

5.1 Seed derivation: binding the work

$$\text{seed} = \text{SHA-256}(\text{protocol} \parallel 0x00 \parallel \text{pre_hash} \parallel \text{nonce})$$

Binding `pre_hash` makes work block-specific: nothing can be precomputed before the parent is known, and nothing can be replayed onto another block. The miner address is not a separate field in the seal; it is committed in the block’s pre-runtime digest, which is part of the pre-image of `pre_hash`. Changing the address changes the digest, hence `pre_hash`, hence the seed, hence the entire asteroid and the work: observing another miner’s candidate nonce is worthless, and the address that seeds the work is the same one that receives the reward.

The seed is the *full* 32-byte digest, and this width is a security parameter, not a convenience. If the seed were truncated, the work domain would be capped at the truncated width, and a capped domain is enumerable: a miner scans it once offline, keeps the seeds whose work clears difficulty, then reaches them by hashing nonces instead of scanning. The advantage of such a table scales as $(N/2^w) \times (C_{\text{scan}}/C_{\text{hash}})$ for table size N and seed width w ; difficulty cancels out of the ratio, so no retarget corrects for it, and the table appreciates with the value of the chain. At $w = 256$ the table cannot be built. The generator chain accepts the full digest end to end (`Pcg64::from_seed`, the noise field, the permutation shuffle), because a full-width seed is only as strong as the narrowest stage that consumes it.

The protocol tag enters the seed for the same reason it enters the work value: two protocol revisions must not share an asteroid space, or generate-and-scan labour spent under one revision would replay under the other for the price of a bucket hash.

5.2 Deterministic asteroid generation

`asteroid()` expands the seed through a PCG64 RNG into the asteroid's scalar parameters (elongation ratios, crater placement) and, critically, re-seeds a fresh fractional-Brownian-motion (fBm) noise field for every seed, then displaces an icosphere refined to subdivision 4 (2,562 vertices). Resolution is pinned by protocol: a miner cannot lower the mesh density, and cannot supply a mesh at all; the generator is the only entry to the pipeline. That kills the trivial-object and forgery attacks of Section 1.3 by construction.

The *work counts* are pinned too: 5 noise octaves, 10 craters, 4 lobes, all protocol constants that the seed cannot vary. If per-seed cost were drawn from the seed, a miner could peek at the cheap parameters first and drop expensive seeds before scanning, paying less than one scan per nonce on average. Constant counts make every nonce cost the same, so there is nothing to filter.

5.3 Spectral scan: the work itself

The mesh is scanned by the `spectral3d` public API [5]: the shape is normalized (translation and scale removed, the features rotation-invariant by construction), its surface sampled at 4,096 points, and condensed into a 23-dimensional spectrum: 2 covariance eigenvalue ratios, a mean-radius statistic, 6 area-weighted and 6 radius-weighted spherical-harmonic power values (orders 1–6), and an 8-bin radius histogram. This is genuine geometric computation (sampling, covariance, spherical harmonics) executed once per nonce; a million attempts are a million scans. The sample count is protocol-pinned, like the mesh resolution.

5.4 Fine quantization: restoring full entropy

$$s = \text{SHA-256}(\text{protocol} \parallel 0x00 \parallel \text{pre_hash} \parallel \text{bkts}), \quad \text{bkts}[i] = \text{round}(\text{feats}[i] / (\text{STEP}[i] \cdot \epsilon_{\text{scale}}))$$

This stage is where the analysis of Section 1.3 lands. STEP is `spectral3d`'s identity-grade bucket width. Its coarseness is what collapses a recognition work space. PoScan multiplies it by $\epsilon_{\text{scale}} = 10^{-5}$ (per-dimension quantum $\approx 10^{-6}$), far below the noise floor, so neighbouring seeds land in different buckets across 19–21 of the 23 dimensions and the bucket vector inherits the full chaos of the per-seed noise field. Hashing the concatenated integer buckets with the protocol string and `pre_hash` yields `s`, uniform over 256 bits. `pre_hash` enters here directly, not only through the seed, so work stays block-specific even if a future generator revision narrows its internal state. A scanned seed table never outlives one block. The difficulty judgment (`s` × difficulty must not overflow U256) is the standard Substrate PoW comparison, unchanged. The protocol string (Appendix A) is a pure version token; any rule change bumps it, throwing old and new work into disjoint spaces (a clean fork point).

5.5 Seal and verification

The seal stays `{nonce, work}`: bucket vectors and meshes never travel. A verifier replays the entire path (derive seed, regenerate mesh, rescan, requantize, rehash) and accepts if and only if the recomputed `s` equals `seal.work` and meets the difficulty. Verification costs exactly one scan, measured at roughly 4 ms native and 6.5 ms inside the wasm runtime. The miner pays millions of them, so PoW's asymmetry survives inside the 5 s import budget. The miner address used in seed derivation is read from the block's author digest at import, so the binding holds without

extending the seal. Verification executes inside the wasm runtime via `PowVerifyApi`, so a runtime upgrade can hot-swap the algorithm.

5.6 Why shortcuts fail

PoScan's work claim is only as strong as the absence of a cheaper path to `s`. Two shortcut families were attacked experimentally and closed by parameter choice; three more are closed by construction.

- **Low-resolution proxies (measured).** At identity-grade buckets, a cheap decimated mesh reproduces the winning bucket vector 65–100% of the time; the scan could be faked. At $\epsilon_{\text{scale}} \leq 10^{-4}$ the hit rate is measured zero across every decimation tried; the floor is set by sampling drift (512 vs 4,096 samples drift $\sim 10^{-5}$), hence $\epsilon \sim 10^{-6}$ with margin.
- **Surrogate models (measured).** Predicting features from the seed's scalar parameters without evaluating the noise field fails because the fBm field is re-cast per seed: swapping only the fBm implementation jitters 19–21 of 23 dimensions beyond the quantum. Computing the features requires evaluating the true field at mesh resolution, which is exactly the work being avoided.
- **Pre-scan tables (by construction).** Scan a seed domain offline once, keep the winners, then mine by hashing nonces into the table, with the scan bypassed entirely and mining security degraded to SHA-256 throughput. Closed twice over: the full 256-bit seed makes the domain unenumerable, and `pre_hash` inside the work hash means whatever table existed dies with the next block.
- **Cost-peeking filters (by construction).** Skip expensive seeds by peeking at cheap generator parameters first. Closed by pinning every work count as a protocol constant (Section 5.2).
- **Self-chosen meshes (by construction).** The generator is seed-locked and verifiers regenerate (Section 5.2).

5.7 Bit-level determinism

`s` enters consensus, so every node must recompute it bit-identically; a bucket-boundary disagreement is a chain split. The scan pipeline is `⋈64` geometry, and the discipline that makes `⋈64` consensus-safe is explicit (Table 1). Verification runs in the wasm32 runtime, which enforces strict IEEE-754 (no x87 excess precision, no implicit fma) and is therefore easier to make deterministic than native, not harder.

Measure	Rule
software floating point	All transcendental math through the <code>libm</code> crate (pure-software, bit-stable); platform <code>libm</code> banned; the generator's single <code>powf</code> call replaced with <code>libm::pow</code> (measured zero drift).
no contraction	<code>fma/mul_add</code> and <code>fast-math</code> disabled; Rust does not auto-contract, and <code>wasm32</code> forbids it structurally.
own noise generator	A self-written classic 3D Perlin + fBm (integer permutation hashes + polynomial interpolation only). The <code>noise</code> crate was dropped: its <code>no_std</code> was a shell that pulled <code>num-traits</code> with <code>std</code> on, breaking the <code>wasm</code> build. Changing the noise type requires a determinism re-audit.
pinned base RNG	PCG64 (<code>rand_pcg</code>) seeded with the full 32-byte digest, with a fixed <code>f64</code> conversion layout (high 53 bits / 2^{53}), as much a determinism lifeline as the noise field.
golden anchors	The golden test anchors the work value <code>s</code> itself (not an intermediate feature), built for <code>wasm32-wasip1</code> against <code>primitive-types</code> (not <code>sp-core</code> , whose <code>secp256k1</code> C build breaks cross-compilation). Any drift on any platform fails the build.
wasm verification	Verification runs in the <code>wasm32</code> runtime; the determinism CI reproduces the same golden <code>s</code> on <code>x86-64</code> and <code>aarch64</code> , natively and under <code>wasmtime</code> .
architecture policy	Mining on <code>x86-64</code> / <code>aarch64</code> ; verification on <code>wasm32</code> ; <code>i686</code> (<code>x87</code>) not supported.

Table 1 – determinism measures for the floating-point scan pipeline.

5.8 One block, one asteroid

Because generation is deterministic, the chain never stores geometry: the winning (`pre_hash`, `nonce`) pair *is* the asteroid, reconstructible bit-exactly by anyone forever. The chain is a catalogue, not a warehouse, and block height doubles as catalogue number. What is stored is minimal; what is shown is derived, so explorers and wallets render any block's asteroid straight from the export RPC (Section 8.1) without special indexing, and the rendering style may improve arbitrarily without changing a single consensus byte.

The artifact is a proof trace and a collectible (Figure 4). Expended work leaves a visible receipt. Optional collectible mechanics (naming, transfer, curation) belong in a future pallet strictly outside the consensus path, where they can add culture without ever being load-bearing for security.

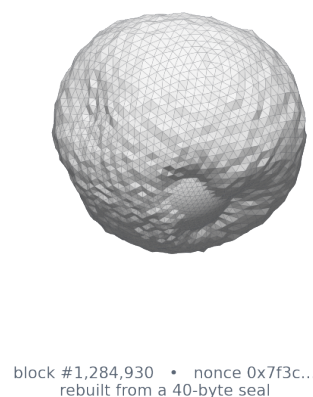


Figure 4 – a single sealed block's artifact, reconstructed from its nonce alone via the export RPC. Rendering style is a presentation choice and never touches consensus.

6 Difficulty regulation: ASERT with outage re-anchoring

6.1 Absolutely scheduled exponential targets

Difficulty follows ASERT [4] (the `aserti3-2d` family): the next target is computed directly from how far the chain is ahead of or behind its absolute schedule, measured against a fixed anchor block.

$$\text{next_target} = \text{anchor_target} \times 2^{(\Delta t - T_{\text{ideal}}) \cdot \Delta h / H}$$

with $T_{\text{ideal}} = 10$ s and half-life $H = 1800$ s (about 180 blocks): every 30 minutes of schedule lag halves difficulty, every 30 minutes of schedule lead doubles it (Figure 5a). Because each target is computed absolutely from the anchor rather than recursively from the parent, the oscillation and cumulative rounding drift of window-based retarget algorithms disappear. All arithmetic is integer fixed-point: 2^x is approximated by a 16-bit fixed-point routine (integer shift plus a cubic polynomial, error < 0.013%) whose coefficients are consensus-frozen, and the target saturates into $[1, U_{256} : \text{MAX}]$ rather than overflowing or reaching zero. Within that range difficulty moves exactly as the schedule demands; there is no per-step clamp, rate limit, or frequency cap to game. The anchor initializes at block #1 (genesis carries no wall-clock timestamp); on schedule, consecutive difficulty changes stay within regular fluctuation bounds.

ASERT trusts timestamps, so timestamps are constrained: a block's timestamp may run at most 2 s ahead of network time, and it must advance at least 2 s past its parent's or the block fails import. Within that window, the worst-case timestamp manipulation moves difficulty by 0.077%, economically irrelevant against the 1,800 s half-life.

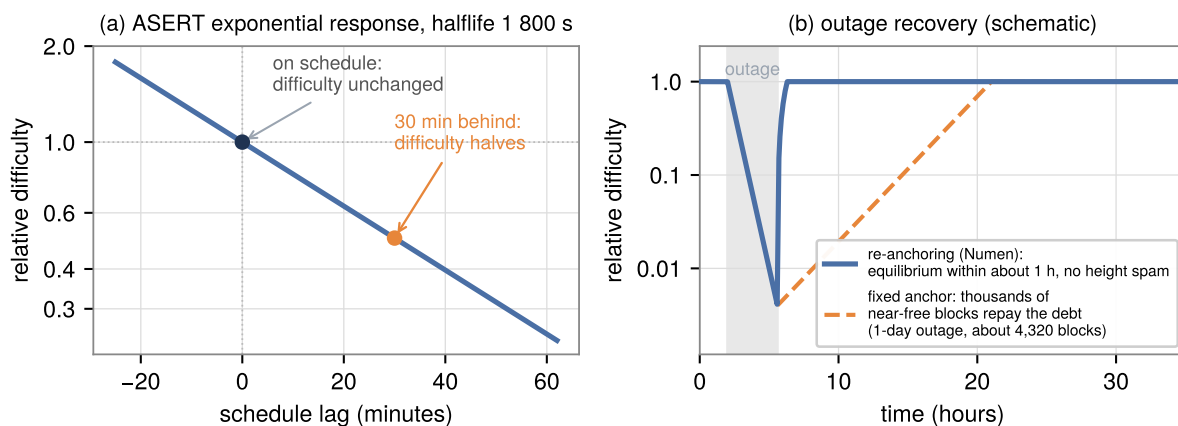


Figure 5 – (a) the exponential schedule response; (b) recovery behaviour after an outage (schematic).

6.2 Recovery from hash-power collapse

Miners query a real-time difficulty that evaluates ASERT against the wall clock, so if hash power vanishes the effective difficulty decays continuously. After 90% of hash power leaves, the chain resumes block production within about an hour (the availability target), with no special-case code. A pure fixed anchor, however, treats downtime as schedule debt to be repaid in blocks: after a one-day outage the chain would owe thousands of near-free blocks. That is height spam with no economic meaning. The protocol therefore *re-anchors on interruption*: whenever the gap between consecutive blocks exceeds 1,800 s, the anchor moves to the recovery block (its target, timestamp, height). Height debt vanishes; if hash power has returned, blocks arrive faster than schedule and ASERT presses difficulty back to equilibrium within regular bounds (Figure 5b). The recovery block itself is mined at the decayed difficulty. The protocol accepts that as bounded

compensation for restarting the chain. Repeated re-anchoring under repeated outages degrades gracefully to parent-relative behaviour. Validation recomputes difficulty from consensus data only (parent state and the block's own timestamp, never the verifier's local clock), so a block's difficulty is the same on every node at any time.

7 GRANDPA and validators

GRANDPA supplies the finality; validators supply the GRANDPA. The algorithm is an agreement, not a shield: every guarantee it makes is borrowed from the permissioned set that does the voting. Validators are the accountable half of the protocol, and this section is about keeping their set sound.

7.1 GRANDPA's fragility

GRANDPA carries no economic weight of its own. Every active validator holds exactly one vote and the set is capped at 1,000, so finality is decided by headcount rather than by stake. The set is therefore the entire attack surface, and it fails in three distinct ways. Lose more than 1/3 of the validators to downtime and no round can reach 2/3, so finality stops while PoW keeps producing blocks. Against 10 honest validators, an attacker who enrolls 5 more and parks them offline does exactly that without casting a vote. Corrupt more than 1/3 and the same arithmetic becomes a safety fault, since two conflicting prefixes can each gather enough votes. Corrupt more than 2/3 and the attacker finalizes whatever it wants, because finality outranks accumulated difficulty and honest nodes will follow. Against the same 10, that takes 21 seats. Hash power cannot repair any of these. What guards the set is who gets in and who stays live.

7.2 Why entry is by invitation

The arithmetic above bites hardest at launch, when the 2/3 that captures finality is a handful of seats. Suppose entry were open, gated by an affordable stake, the standard recipe. Mining is the only issuance, so in the chain's first years the large holders are, by construction, the miner interest bloc; whatever a seat costs, the mining bloc is the first and for a long time the only party able to pay for enough of them. A bloc that locks its way to 2/3 of a small set holds both halves of consensus at once. It produces the blocks and it finalizes them, and a group that writes history and also ratifies it can settle whatever ledger it likes.

Numen closes this from three sides. Staked entry is closed for the early years, and closed the simple way: no switch and no special mechanism, just a lock threshold set beyond capture, with a capturing share of seats costing more than the entire genesis issuance. The working entrance is an invitation instead: centrally reviewed, deliberately so, the same passage many networks made through a permissioned early phase. And the airdrop works the distribution side, spreading vote-weight across a far broader group (Section 11).

7.3 Is the invitation era another monopoly?

Call it what it is: centralized control over who validates. Two facts bound what that control can do. First, entry control is a security parameter, not an economic one; the gate decides who may vote in GRANDPA and touches nothing else, no balance, no referendum, no identity. Second, the process is public: the criteria below are published, anyone who meets them can apply in the open, and every grant and removal on the invitation list is an on-chain event the community can

audit. The gate exists to stop a monopoly on finality, not to hand `prime` one, and `motoz` commits to never abusing it.

If the word “centralized” still repels you, that instinct is healthy. But weigh it against Section 7.2: an open door at launch does not produce decentralization, it produces the surface of one, `referenda` and finality both held in fact by the bloc with the coins. You must choose: trust the founder, or trust the bloc. When the invitation era ends is Section 13.3.

7.4 Who gets invited

Entry has two gates, both checked on-chain. First, a *qualified on-chain identity* (Section 10), meaning a judged identity with a reachable plaintext social channel. Second, an invitation from `prime`. Provided the set is not full and the account is not in a cooldown, the candidate joins the active set at the next session boundary.

`prime` looks for four things in a candidate.

- An active member of the community.
- Trusted by the community.
- Unaffiliated with mining pools and large miners.
- Willing to pay for a well-connected server.

`prime` maintains an on-chain *stake-exemption list* (`set_stake_exempt`, every change a public event): listed accounts join with a zero-amount lock, and the other gates (identity, session key, capacity, cooldown) stay in force. Genesis validators are enrolled automatically at genesis and may re-enter stake-free until removed; an exempt validator’s lifecycle is otherwise identical to a staked one’s, so the machinery has no second code path. The exemption doubles as liveness insurance against the 1/3 stall of Section 7.1, since the list lets the project backfill validators in an incident without friction. It touches nothing else; exempt validators cannot spend, cannot slash, cannot alter any account.

7.5 Two keys, never mixed

A validator operates two keys with sharply separate roles. The *account key* holds the funds, carries the stake and the on-chain identity, and takes any consequences; it signs only three on-chain actions, registering session keys, locking to join, and requesting exit. It stays offline, never on the server. The *session keys* (GRANDPA and im-online) are the working keys the node signs with continuously; the node generates them on the server and their private halves never leave it. They hold no assets: a compromised server loses only the “mask,” and the operator re-registers a fresh set with the offline account key while the stake stays untouched. This is why a mainnet validator can run on modest hardware while keeping custody of a large lock entirely offline.

7.6 Liveness enforcement

Every session, each validator must broadcast a signed heartbeat. Missing a single session’s heartbeat (about 10 minutes) triggers automatic eviction at the next boundary (Figure 6). That rule is deliberately strict because a 10-second-block chain cannot afford a drowsy finality committee. Detected GRANDPA equivocation (double-voting in prevote or precommit) triggers the same eviction path, with the equivocation record kept public and permanent.

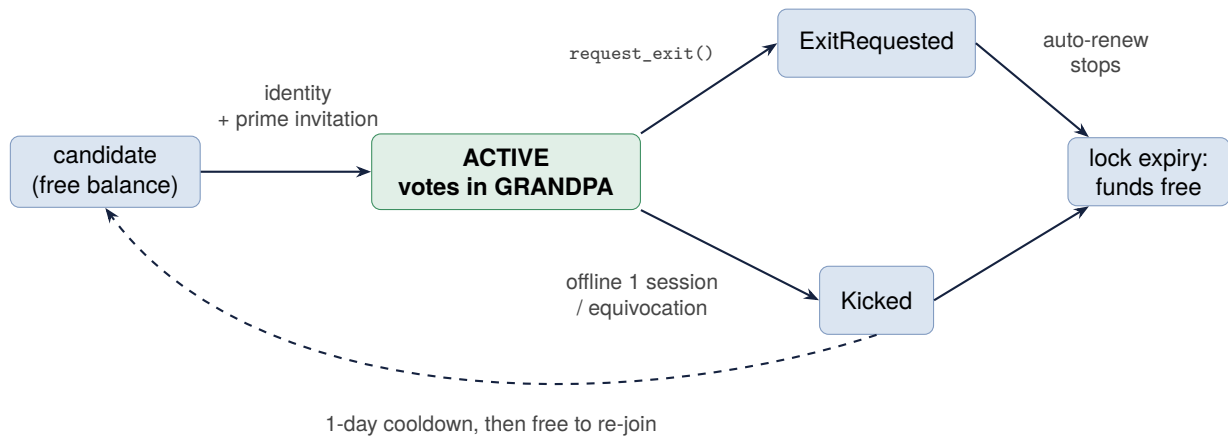


Figure 6 – validator lifecycle. All transitions take effect at session boundaries. There is no slashing; locked funds are never destroyed, only time-locked to their original expiry.

7.7 No slashing and no reward, by design

Neither offline eviction nor equivocation destroys locked funds, and validators earn no protocol reward: block rewards go to miners, and fees split between miners and the treasury. These two facts are one deliberate design, not two concessions.

No reward is what makes no-slashing coherent. If validating paid a yield, the network would attract operators chasing that yield. They would cut node specifications to maximize margin and run finality on the cheapest hardware available. A finality committee of under-provisioned, profit-trimmed boxes is exactly what a 10-second chain cannot tolerate. Removing the reward removes that incentive at the root: the only people who lock a large sum for months to run demanding infrastructure with no financial return are those who want the network to work. Running a validator takes real operational skill; pricing it in altruism rather than yield filters for that skill and against the race to the bottom. And with nothing to earn, there is nothing a slash needs to protect, so no honest operator can lose capital to a misconfigured firewall or a transient outage.

The deterrents are therefore exclusion and time, not confiscation: an evicted validator stops earning influence immediately, keeps its coins locked until the original expiry, and cannot re-join for a one-day cooldown. This trades the sharper crypto-economic edge of slashing for a materially lower barrier to honest participation, which suits a young network whose finality-safety margin is capacity, not stake-at-risk. Finality safety still requires that less than 1/3 of validator weight be malicious. As the network matures, fee-sharing or explicit finality rewards may be revisited through governance, as an honest “not yet” rather than a permanent “never.”

7.8 Lock mechanics

Locked coins cannot be transferred or unlocked before expiry. For active validators the system renews the lock automatically once per day, so honest operation never lapses; eviction or a voluntary `request_exit()` stops auto-renewal, after which the lock simply runs out and the funds free themselves. There is no early unlock, even after exit, and the same holds for a candidate who locked but never entered the set (session key missing, or the set full): the lock runs to its original expiry with auto-renewal stopped, never refunded early. Re-joining after an exit is always a fresh lock under the standard conditions; where an old record exists, the stricter value wins; the unlock height only moves later, and the locked amount only grows.

8 Mining service: built-in and external

Mining runs in two modes [6], selected by node configuration, sharing one non-negotiable rule: **every seal is verified by full PoScan replay before import; external compute is never trusted** (Figure 7).

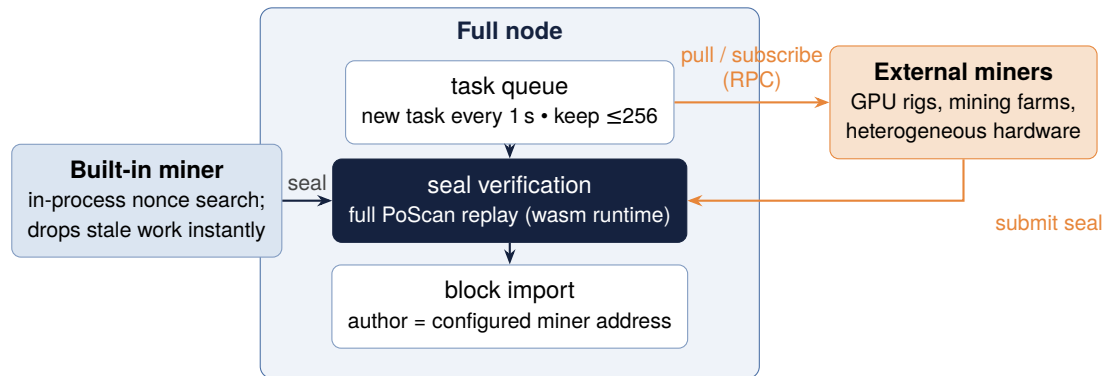


Figure 7 – built-in mining inside the node process, and external mining over the `mining_*` RPC. Every submission is re-verified by full replay; difficulty is always recomputed from the parent block, never taken from the task.

Two switches select the mode and can coexist. `--miner <SS58>` enables mining and always exposes the `mining_*` RPC (external mining available by default); `--node-miner [threads]` adds the in-process scan loop on top. `--miner` alone is external-only; the two together coexist; the default is not to mine, and `--node-miner` without `--miner` is an error.

Both modes draw from one task queue. A new task is cut every second, up to 256 retained so slightly stale solutions remain submittable; all tasks are cleared when a new block arrives, and submissions against an outdated `pre_hash` are rejected. A task carries everything the work function needs: `pre_hash`, target difficulty, the bound miner address, and the protocol identifier.

- **Built-in.** The node runs the nonce search in-process, stamps its configured miner address as block author, splits the nonce space across a configurable number of threads by stride (default: all cores), and abandons stale work the moment the best chain head moves, so no cycles are burned on dead parents. This mode exists for testing and benchmarking. Competitive hash power belongs on external hardware.
- **External.** The node publishes those tasks over the `mining_*` namespace: `mining_getTask` to pull, `mining_subscribeTask` to be pushed every new task, `mining_submitSeal` to return a solution. This is the doorway for GPU rigs and heterogeneous hardware; exposure of the RPC surface is the operator's choice.

Two hardening details close the obvious attacks. *Difficulty is decoupled from tasks:* seal verification and total-difficulty accounting always recompute the live difficulty from the parent block, never trusting the value a task or submission carries; miners bear the risk of difficulty moving mid-search. *Rewards are pinned:* a block reward goes to the node's configured miner address, which is also the address the pre-runtime digest commits to, so an external worker cannot redirect it. Pool-style payout splitting is out of scope for this version.

8.1 Reconstructing any block's asteroid

Because a block records only its nonce, the block's asteroid is regenerated on demand rather than stored. A read-only RPC (`poscan_getObject`) takes a block hash and returns the OBJ text of

that block's asteroid. Critically, the geometry is recomputed through a runtime API on the block's *parent runtime*, reusing the exact generator that mining and verification use, so a historical block reconstructs bit-faithfully even across a later algorithm upgrade. The node only disassembles the block (recovering the nonce from the seal digest and the `pre_hash`) and formats OBJ text; it trusts no caller-supplied parameters. Any node can serve the export, miner or not; faithfully reconstructing arbitrary history requires an archive node that retains the corresponding state.

9 EVM compatibility

Numen embeds Frontier [3] (`pallet-evm` + `pallet-ethereum`), targeting the Osaka EVM ruleset. Solidity bytecode deploys unchanged; MetaMask connects with a chain-id entry; Hardhat and Foundry work against the standard Ethereum JSON-RPC surface (`eth_*`, `net_*`, `web3_*`) served alongside Substrate RPC. The native NUMN uses 18 decimals, matching Ethereum conventions exactly. Substrate AccountId and EVM H160 addresses are bridged by a deterministic mapping with two-way transfers, so one balance serves both worlds.

Chain-ids are split per network (mainnet 32026, testnet 320261, development 320262), so an EIP-155 signature cannot be replayed across networks. Two fee markets run in parallel, each pricing its own transactions: EVM transactions use an EIP-1559 dynamic base fee (`pallet-base-fee`, initial 1000 gwei, elasticity 12.5%, target/upper block fullness 50%/100%), while Substrate transactions use a slow-adjusting multiplier calibrated to the same price level (Section 11.3).

9.1 Native-token ERC20 precompile

The native coin is exposed as a first-class ERC20 at the fixed address `0x0000...0802`, so contracts and tooling can treat NUMN exactly like a token without a wrapper deployment.

- **Standard facade.** `name` / `symbol` / `decimals` / `totalSupply` / `balanceOf` / `transfer` / `approve` / `transferFrom` / `allowance`, emitting `Transfer` and `Approval` logs. Allowances and nonces are independent on-chain state, kept out of the runtime metadata.
- **Cross-VM withdrawal.** `withdraw(bytes32,uint256)` carries an H160 → AccountId transfer, completing the two-way bridge on the EVM side.
- **WETH9-style deposit.** A value-bearing empty call, the `deposit` selector, or any unknown selector with value is treated as a deposit and refunded at par (net balance unchanged), emitting a `Deposit` log; a zero-value call reverts. Contracts written against WETH9 work unmodified.
- **EIP-2612 permit.** Off-chain EIP-712 signed approvals with deadline and a replay-guarding nonce, exposing `nonces()` and `DOMAIN_SEPARATOR()` so third parties can submit approvals gaslessly.
- **Real-caller charging.** The precompile debits the actual caller and cannot be borrowed: it reverts any `DELEGATECALL` or `CALLCODE` outright, so a malicious contract cannot run the transfer logic under a victim's storage context, while ordinary `CALLs` from EOAs, contracts, and other precompiles pass. The standard precompiles 1–8 keep stock Ethereum behaviour.

10 Account features

Numen ships the Substrate account primitives that teams and power users expect, and its own governance is built on the same ones; the treasury proxies are not special cases but ordinary uses of these pallets. All storage-consuming registrations price their deposits from one constant: the *permanent storage byte price* of 0.01 NUMN per byte.

- **Multisig.** An m-of-n shared account whose address is derived deterministically from the member set and threshold; a call executes once a threshold of members approves, with an in-progress proposal held on-chain against a refundable deposit (base 5 NUMN plus 0.32 NUMN per signatory, up to 100 signatories).
- **Proxy.** An account may authorize proxies to act for it under a typed scope: `Any`, `NonTransfer` (excludes balance transfers, EVM entry points, and vested transfers), or `Governance` (treasury, bounty, and referendum calls, including their utility batches). Keyless pure proxies and time-delayed announced execution are supported; registrations and announcements are held against per-item deposits.
- **Batch and derivation.** `batch` executes several calls in one transaction (stop-on-error, no rollback); `batch_all` is atomic (any failure reverts the whole set); `as_derivative` lets one key act as any of its indexed sub-accounts.
- **On-chain identity.** An account may register an identity (display name, social channels, contact fields), holding a deposit per encoded byte, and attach up to 100 sub-accounts that inherit its qualification. A registrar set (capacity 1000) issues judgments on identities. A *qualified identity* is one judged `Reasonable` or `KnownGood` that also carries at least one plaintext social channel among X, Telegram, and Discord. Referendum submission and validator entry reuse this same gate. It sets an accountability floor, so a counterparty can always reach someone. Registrar appointment is an administrative origin that `prime` shares with the identity admin track (Section 12), username authorities answer to that track alone, and clearing someone else's identity sits behind `ForceOrigin`, which only a root referendum reaches.
- **Vesting.** Balances may be granted under a per-block linear release schedule. The unreleased portion is fenced; it cannot be transferred, cannot serve as a deposit, and cannot pay fees; it becomes spendable only as it vests. Vested transfers have a 1 NUMN minimum and an account holds at most 28 concurrent schedules.

11 Tokenomics and monetary policy

NUMN has a hard cap of 1,000,000,000 tokens, issued in two parts: a one-time genesis issuance of 600,000,000 and a mining issuance of 400,000,000 that a halving curve approaches asymptotically but never exceeds (Figure 8).

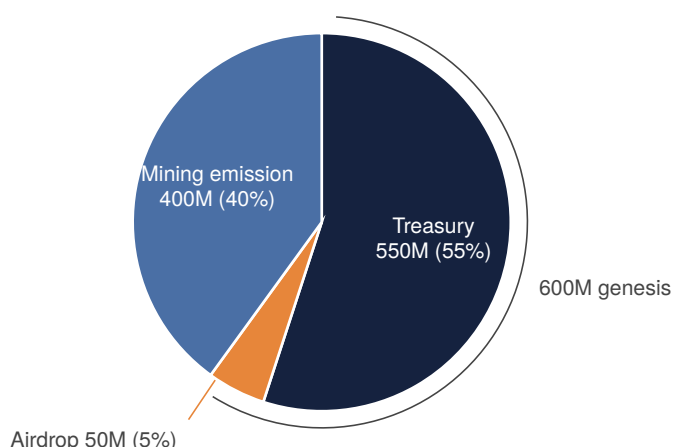


Figure 8 – issuance structure. Genesis (600M) is split between a community-governed treasury and a launch airdrop; the remaining 400M is emitted to miners over decades.

11.1 Genesis issuance: no team pre-mine

The 600,000,000 minted at genesis contains **no pre-funded team or investor account**. It is split two ways:

- **Treasury: 550,000,000 NUMN.** Held by an on-chain treasury account (Section 12), spendable only through governance and topped up by the treasury's fee share (Section 11.3). It funds the ordinary work of a public chain: marketing, ecosystem growth, developer grants, and bounties. It is not a team wallet; no single party can spend it.
- **Airdrop: 50,000,000 NUMN.** Reserved as a launch warm-up distribution to seed a wide holder base and the initial governance float, held at a published genesis address (Appendix A). It is paid to registered addresses as vested transfers and unlocks linearly from genesis over about two years.

11.2 Mining issuance: a halving curve to a fixed ceiling

Miners are paid a block reward that starts at 16 NUMN and halves every 12,500,000 blocks (about four years at 10-second blocks) through the sequence $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow \dots$. The geometric series converges exactly:

$$\sum_{e=0}^{\infty} \frac{16}{2^e} \times 12,500,000 = 16 \times 12,500,000 \times 2 = 400,000,000 \text{ NUMN.}$$

Because the sum is bounded by construction, no on-chain total-supply check is ever needed; the curve simply cannot overshoot 400M (Figure 9). Combined with the genesis 600M, total supply rises from 600M toward the 1B ceiling, reaching roughly 800M by year 4 and 900M by year 8, asymptotically approaching but never reaching 1B.

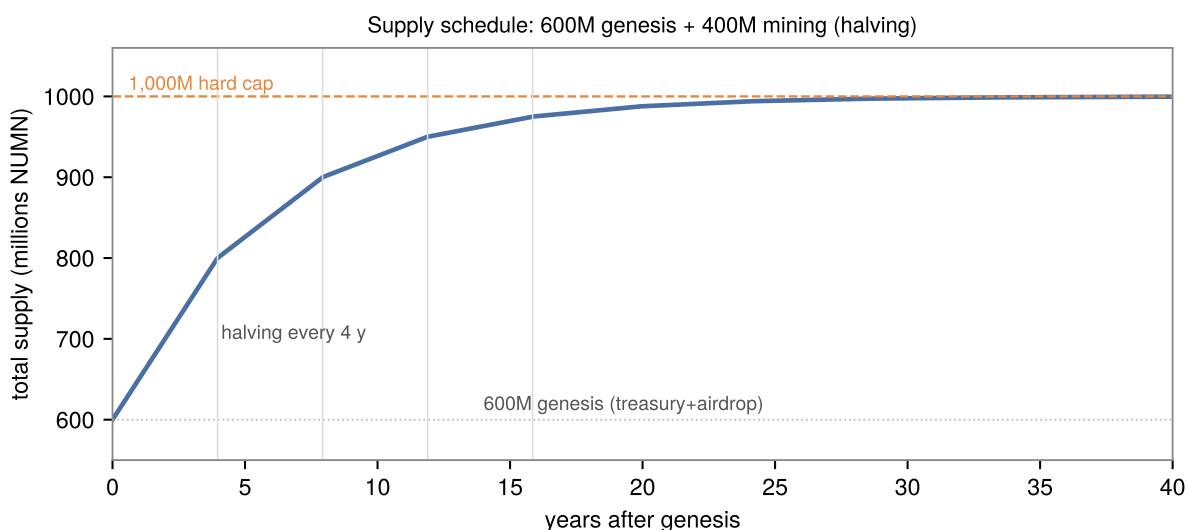


Figure 9 – supply schedule. A fixed cap makes the long-run transition to a fee-funded security budget explicit, rather than hiding it behind perpetual tail emission.

11.3 Fees, and where value accrues

Transaction fees (Substrate fees and the EVM base fee) split 20/80 between the block author and the treasury, tips reach the author whole, and a block produced without a miner digest pays everything to the treasury; nothing burns. Miners thus take new issuance, a fee share and tips,

and validators take nothing (Section 7). Security spending concentrates on the permissionless, capital-free role, while finality runs on opportunity cost.

The two fee markets are calibrated to one price level (Table 2). At the EVM's initial 1000 gwei base fee, one unit of EVM gas corresponds to 20,000 units of Substrate weight, and the Substrate length fee prices a calldata byte at the Ethereum-conventional 16 gas, so the same operation costs the same in either world, and neither side becomes the discount door. The Substrate multiplier adjusts slowly with block fullness (target 25%) and is clamped to a $0.1\times$ – $10\times$ band: persistent full blocks take under three days to reach the ceiling, and the floor cannot decay to dust while the chain idles pre-adoption. Permanent storage is priced separately from execution. One constant, 0.01 NUMN per byte, drives every on-chain deposit (preimage, multisig, proxy, identity, bounty metadata), so “forever bytes” have one price everywhere.

Parameter	Value	Notes
Total supply cap	1,000,000,000 NUMN	fixed; issuance = genesis + mining
Genesis treasury	550,000,000 NUMN	governance-spendable only
Genesis airdrop	50,000,000 NUMN	launch distribution, linear vesting over ≈ 2 y
Mining issuance	400,000,000 NUMN	halving curve, geometric convergence
Initial block reward	16 NUMN	first halving epoch
Halving period	12,500,000 blocks (≈ 4 y)	sequence $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$
Orphan / uncle reward	none	clean incentives at 10 s + fast finality
Transaction fees	20% author, 80% treasury	tips fully to the author
Fee parity	1 gas $\equiv$ 20,000 weight @ 1000 gwei	calldata byte = 16 gas equivalent
Fee multiplier band	$0.1\times - 10\times$	slow-adjusting, 25% target fullness
Storage byte price	0.01 NUMN / byte	single source for all deposits
Existential deposit	0	no dust reaping
Decimals	18	1 NUMN = 10^{18} base units

Table 2 – economic parameters.

12 Governance and treasury

12.1 OpenGov and the treasury

Numen runs OpenGov (`pallet-referenda` with `pallet-conviction-voting`). Holders vote on treasury spending, on runtime code, on who may judge identities and grant usernames, and on whether another referendum gets to finish. Voters lock for conviction-weighted power ($1\times$ to $6\times$, with a 7-day base lock) and may delegate per track; proposal call-bodies are stored via `pallet-preimage` and executed by `pallet-scheduler` at the enactment block. Submitting a referendum requires a qualified identity (Section 10) and a 5 NUMN deposit: a judged, reachable identity stands behind every proposal. That prices out drive-by spam without pricing out people. Nine tracks carry the vote, each with its own periods, deposit, and pass curves (Table 3). Spending is split across three `Spender` tracks by per-transaction ceiling.

Track	Cap	Prepare	Decide	Confirm	Enact	Dep.	Conc.
root	none	1 d	28 d	1 d	1 d	100K	1
runtime_upgrade	none	10 min	28 d	10 min	10 min	100	1
wish_for_change	none	2 h	28 d	1 d	10 min	1K	10
identity_admin	none	2 h	28 d	1 d	10 min	1K	10
referendum_canceller	none	2 h	7 d	1 d	10 min	1K	1000
referendum_killer	none	2 h	28 d	1 d	10 min	10K	1000
small_spender	200K	4 h	28 d	1 d	1 d	100	100
medium_spender	1M	4 h	28 d	3 d	1 d	200	20
big_spender	10M	4 h	28 d	7 d	1 d	1K	2

Table 3 – the nine referendum tracks. Caps and deposits are in NUMN. Approval everywhere opens at 100% and settles at 50%. Root, the upgrade track, wish for change and identity admin ease down a reciprocal, passing 80% a seventh of the way through; the rest fall straight, the spenders landing on days 7, 14 and 28 and the brakes at the close of their decision period. Support everywhere opens at 50%. Root and wish for change run a straight line to nothing. The others take reciprocals, the upgrade track through 1% inside a day, identity admin and both brakes through 2% at three sevenths of the way and on to nothing, and the spenders down to floors of 0.5%, 0.75% and 1%. Where support bottoms out at zero the confirm period sets the real bar.

Support (turnout) thresholds are measured against *active issuance*, not total supply: the treasury's own holdings are marked as deactivated issuance each block and excluded from the denominator, so the coin that does not vote cannot dilute the voters' support ratio. At genesis the denominator is roughly the 50M airdrop float and grows with mining. Because a treasury payout moves funds from deactivated to active, the more the treasury spends, the larger the denominator and the higher the bar for the next spend. The damper is built in.

12.2 Treasury culture: deliver, then pay

Community voting decides treasury spending, but voters are not auditors and cannot catch every scam at the ballot. Numen therefore leans on structure rather than voter vigilance. Payouts flow through bounties and child-bounties: a proposal is approved for a scope, a curator is appointed against a forfeitable bond, work is delivered and verified, and only then are funds released. Deliver first, pay later. Curators can open child-bounties within a parent's budget and appoint sub-curators without returning to a full referendum, so large programmes scale without re-voting every line item, while the per-track ceilings of Table 3 cap the blast radius of any single mistake. Risk is contained by how the funds move, not by assuming the crowd is expert.

13 Sudo and prime

`pallet-sudo` is the last centralized key in a Substrate runtime: it can dispatch anything as Root, and for as long as it exists, on-chain governance is decorative. Numen removes it outright. What remains is a handful of downgraded privileges kept for early-network safety, called `prime`. A single prime key is written at genesis and can be rotated by its holder via `set_key`, which emits an on-chain event. Most of what prime holds is shared with a referendum track, so prime is the fast path rather than the only one. The treasury veto, the validator stake-exemption list, and the right to table a runtime upgrade answer to prime alone.

13.1 Prime's privileges

1. **Runtime upgrade.** prime alone may table a referendum on the runtime-upgrade track. That track's origin is the only caller `upgrade` accepts, and the call forwards `System::set_code` with `Root` authority.
2. **Brakes.** prime is wired to referenda's `CancelOrigin` and `KillOrigin` beside the canceller and killer tracks, and it holds the treasury's `RejectOrigin` alone, which bounties and child bounties inherit. The grounds for a brake are any of the following.
 - an attack on the chain
 - an attempt to exploit an implementation bug
 - a withdrawal request from the submitter
3. **Gates.** prime manages the validator stake-exemption list (Section 7.4) and shares the identity system's registrar origin with the identity admin track (Section 10).

Table 4 takes them one by one and says which ones holders reach by referendum.

Power	Prime	Holders
Open a vote on new runtime code	✓	✗
Replace the runtime code	✗	✓
Cancel a running referendum, deposits refunded	✓	✓
Kill a running referendum, deposits slashed	✓	✓
Void a treasury spend before it is claimed	✓	✗
Drop a treasury payout from the queue	✓	✗
Close a bounty	✓	✗
Remove a bounty curator, bond slashed	✓	✗
Close a child bounty	✓	✗
Remove a child bounty curator, bond slashed	✓	✗
Approve an account to join the validator set	✓	✗
Appoint an identity registrar	✓	✓
Remove an identity registrar	✓	✓
Hand the prime key to another account	✓	✗

Table 4 – who holds each power. Holders act through a referendum.

13.2 Why these privileges sit with prime

Every class of prime privilege invites the same objection: surely this belongs to a referendum. The answer differs per class, so we take them in turn, stating the reasons plainly rather than hiding them.

Runtime upgrade.

1. **The runtime has one author.** Numen's runtime comes from one repository, and nobody else ships one. Opening submission to everyone adds no proposer worth having, only a standing invitation to put hostile code on the ballot. prime puts up what the project built.
2. **Holders decide whether it lands.** The code goes on public record and holders have the decision period to vote it down. prime opens the vote and nothing more.

The brakes.

Cancel and kill sit with prime, and holders reach both by referendum. prime's hand is the fast one. Neither an attack nor a live exploit waits out a vote.

The gates.

1. **Validator invitations.** Why entry is invitation-gated while the authority set is small is Section 7.2.
2. **Identity registrars.** Submitting a referendum takes a qualified identity, and only a registrar's judgement makes one. Genesis ships no registrar, so until prime appoints the first one there is no qualified identity, no referendum, and no way to vote a registrar in. Upstream `pallet-identity` puts appointment and removal behind one origin, so prime holds removal by inheritance, not by design. prime makes one promise. It appoints a single registrar, the service that judges identities automatically. After that it touches `pallet-identity` no further.

13.3 The path to decentralization is left open

None of this is a permanent settlement. Moving the upgrade privilege off prime now takes only a submission gate, since the track already runs and opening it past prime is a one-line origin change. The validator gate opens the same way. As the set grows, governance lowers the 100M entry threshold until invitations stop mattering (Section 7.4). The road is left paved so that decentralizing later is a small patch, not a rewrite.

Each privilege has its own floor conditions. Meeting them opens the handover discussion, not the handover itself; the judgment that no metric can capture, whether vote-weight has genuinely dispersed, stays in the open discussion where it belongs.

- **Opening the upgrade track.** Submission goes to a foundation or a standing technical body once one exists. No date is set.
- **Retiring prime's brakes.** The community puts it to an on-chain vote and decides.
- **The validator gate.** One month of mainnet operation, a healthy PoW layer, and a large enough validator count. A healthy PoW layer means hash rate spread widely enough that no single miner or pool comes near 50%. How large the validator count must be is still open; it needs discussion with security experts in the community, and anything below 10 is out of the question.
- **Identity registrars.** prime loses the privilege at the first runtime upgrade after it appoints the automatic registrar.
- **The rest.** The treasury and bounty vetoes sit with prime only because removing sudo left those origins without a holder. Nothing uses them today. The day something does, a new track takes them and prime steps back.

13.4 Who holds prime

prime is held by motoz. Its mainnet address is in Appendix A and verifiable on-chain. The holder rotates it through `prime.set_key`, which emits a public event.

14 Security analysis

14.1 Propagation and the orphan budget

At a 10-second target interval, the expected orphan rate under propagation delay d is approximately $1 - e^{-d/T}$: about 9.5% at 1 s, 18.1% at 2 s, 25.9% at 3 s, and 39.3% at 5 s (Figure 10). This is the honest cost of choosing 10-second blocks over a slower target: the 15% design ceiling is met only when propagation stays below roughly 2 seconds. That is a real constraint, not a rounding detail.

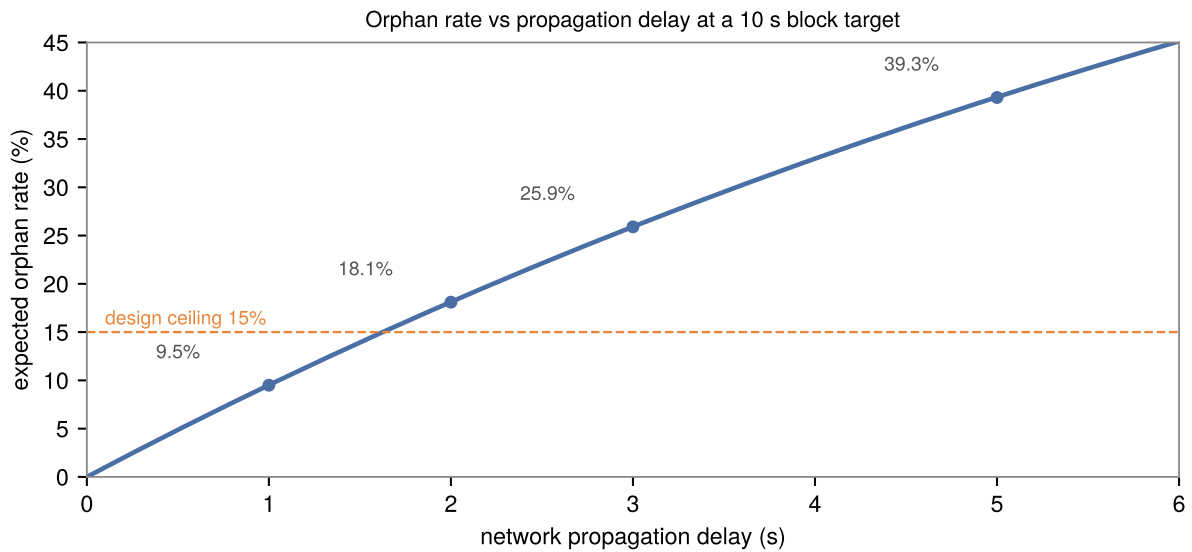


Figure 10 – orphan-rate budget against propagation delay at a 10 s target. Ten-second blocks demand well-connected nodes; the budget tightens sharply beyond 2 s.

Three mechanisms defend the budget. The 2 s block-construction cap and the 5 s import budget bound the self-inflicted portion of delay; the first-seen tie-break removes the incentive to sit on blocks; and serious miners are expected to run near backbone exchange points, where inter-node propagation is well under a second. Fast finality also limits the damage: with no uncle rewards, an orphaned block simply earns nothing, so connectivity is priced directly into miner revenue rather than socialized. Operators who mine or validate should treat network placement (a host genuinely on a Tier-1 backbone at a major IXP, not an edge site) as part of the job.

14.2 Attack surface

Table 5 collects the attacks this design has to survive and the mechanism that answers each one.

Attack	Requirement	Blast radius	Response
majority hash power	>50% of hash power	reorg of unfinalized blocks only	finalized prefix untouchable; finality caps the damage window at ~30 s
validator collusion	>1/3 validator weight	finality stalls (liveness)	PoW production continues; chain grows; finality resumes when weight recovers
finality capture by stake	2/3 of the validator set	finalize an attacker fork (full capture)	100M lock: seven seats cost more than the entire genesis issuance; invitation-era exemption list; identity gate on every entrant
combined attack	>50% hash + >1/3 validators	reorg + stall	requires breaching both independent layers at once
selfish mining	large hash share	profit inside unfinalized window	first-seen tie-break + ~30 s finality shrink the game
timestamp manipulation	block author	difficulty nudging	± 2 s bound; negligible against 1,800 s halflife
equivocation	validator key	conflicting finality votes	detection, eviction, cooldown; public evidence
validator downtime	—	finality latency	one-session heartbeat eviction
PoScan forgery / degenerate mesh	miner	fake or free work	seed-locked generation; verifiers regenerate (Sections 5.2, 5.7)
PoScan shortcuts	miner	cheaper-than-scan work	fine quantization + per-seed noise field; closed experimentally (Section 5.6)
pre-scan seed table	enumerable seed domain	scan bypassed	full 256-bit seed domain (Section 5.6)

Table 5 – attack surface and responses.

14.3 Known risks

We state the open problems as plainly as the design.

- **No FPGA/ASIC resistance in v1 (accepted).** PoScan is compute-bound with small state; specialized hardware will outperform GPUs. v1 accepts this openly. If concentration threatens, a v2 memory-hard layer (an Autolykos-style [7] multi-GB table with random access) is a separate, planned design; the runtime-upgradeable verifier makes the switch forkless in mechanism, though it remains a hard fork in work semantics. One distinction matters here: hardware that *scans faster* runs an efficiency race we accept, while a path that *skips scanning* would break the work function outright. The pre-scan table was the latter, and it is closed by seed width (Section 5.6); no future revision may re-narrow the seed.
- **Centralized early validator entry (transitional).** During the invitation era, who validates is decided by the prime-held exemption list. This is deliberate (the alternative at launch is a finality set purchasable for a rounding error of genesis), but it is a centralization of liveness, stated as such. The exit is the governance-lowered threshold of Section 13.3.
- **Integration seams (tested).** The finality-respecting fork choice must be airtight in the import pipeline; set handovers at session boundaries must let the old GRANDPA set finish its final round; an empty validator set degrades the chain to pure PoW until new locks arrive. Each seam has dedicated tests.
- **PoW algorithm replacement is a hard fork (accepted).** Even via runtime upgrade, switching work semantics partitions miners; activation heights must be planned and announced. The protocol-string mechanism (Section 5.4) makes the partition explicit and clean.
- **prime key custody (live risk).** Enough compromised signer keys put arbitrary code on the ballot (Section 13.2), and the same custody protects the gate privileges (validator invitations,

identity registrars). Holders get the decision period to vote a hostile upgrade down, thin cover when turnout is low. Public rotation events make a takeover visible without preventing it.

- **The long-run security budget is unsolved (open question).** The block reward halves toward zero, so decades out the security budget rests on transaction fees alone. Whether fee revenue is enough to hold the hash power that protects the chain is unknown, and no fixed-cap Proof-of-Work design has answered it. The fixed cap keeps the question in the open instead of burying it under perpetual tail emission (Figure 9), but keeping it visible is not the same as solving it.

15 Risk factors

The following risks are material and stated without softening.

- **Immature software, no audit.** The protocol is under active development, and no third-party security audit has been performed. Bugs, including consensus-level bugs, are possible.
- **Mining centralization.** v1 does not resist FPGA/ASIC hardware, so hash power may concentrate and leave the PoW layer less decentralized than commodity mining would (Section 14.3).
- **Finality depends on altruistic, initially invited validators.** Validators earn no reward and early entry is by invitation, so finality liveness rests on enough capable operators choosing to serve. Too few, and finality stalls while the chain continues as pure PoW (Section 14.3).
- **Upgrade-key compromise.** Compromise of enough prime signer keys would put arbitrary code on the ballot. The attacker still has to win the referendum, so the exposure is a thinly attended vote rather than a signature, and no mitigation removes that risk entirely (Section 14.3).
- **Identity is public by design.** Referendum submitters and validators must expose a judged identity with a plaintext social channel. That accountability floor is deliberate; participants who require anonymity should mine or run full nodes instead.
- **Market and liquidity.** NUMN may have volatile or no market value.
- **Regulatory and legal uncertainty.** Treatment of tokens and public blockchains varies by jurisdiction and may change adversely. The project is maintained anonymously; there is no company standing behind the network and no entity from which to seek recourse.

16 Disclaimer

This document is for information only. It is not investment, financial, legal, or tax advice, and is neither an offer to sell nor a solicitation to buy any asset. It contains forward-looking statements and figures that may change without notice; nothing here is a promise of future performance, value, or delivery. NUMN is a network utility. Holders should expect no profit from the efforts of others. Participate only after your own research and, where appropriate, professional advice.

17 Glossary

PoScan (Proof of Scan)

Numen's PoW: each nonce deterministically generates a 3D asteroid and scans it; the scan is the work.

spectral3d

The rotation/translation/scale-invariant scan pipeline that turns a mesh into a 23-dimensional spectral feature vector.

avalanche

The property that any tiny change of input produces a wildly different output. Required for mining; the opposite of recognition's "similar in, similar out."

GRANDPA

The Substrate BFT finality gadget; validators vote on chains and finalize whole prefixes at once.

equivocation

A validator signing two conflicting votes in one round. The misbehaviour is provable and triggers eviction.

seal

The proof attached to a block header; in Numen, {nonce, work}.

seed

The full 32-byte digest SHA-256(protocol || 0x00 || pre_hash || nonce) that deterministically selects the asteroid a nonce must scan.

session

A ~10-minute epoch (60 blocks) during which the validator set is fixed; changes apply at boundaries.

ASERT

Absolutely Scheduled Exponentially Rising Targets; the anchor-relative, integer-only difficulty rule.

prime

The upgrade-and-brake pallet; a key with stop and gate privileges, able to table an upgrade referendum but not to spend or to ship code.

qualified identity

An on-chain identity judged `Reasonable` or `KnownGood` that carries a plaintext X / Telegram / Discord channel; required to submit referenda and to validate.

stake-exemption list

The prime-managed on-chain list whose members may join validation with a zero-amount lock; the invitation mechanism of the early network.

conviction voting

OpenGov mechanism where longer token lock-up grants proportionally more voting weight (up to 6x).

existential deposit

The minimum balance below which an account is reaped as dust.

protocol string

The versioned identifier of the PoScan rule set (Appendix A); any change bumps it and forks work into a disjoint space.

References

- [1] Polkadot SDK / Substrate documentation. docs.polkadot.com.
- [2] A. Stewart, E. Kokoris-Kogias. *GRANDPA: a Byzantine Finality Gadget*. arXiv:2007.01560.
- [3] Frontier, the Ethereum compatibility layer for Substrate. github.com/polkadot-evm/frontier.
- [4] J. Toomim, M. B. Lundeberg. *aserti3-2d absolutely-scheduled difficulty adjustment (ASERT)*. Bitcoin Cash, November 2020 upgrade specification.

- [5] spectral3d, rotation/translation/scale-invariant 3D shape spectra.
github.com/numen-network/spectral3d.
- [6] Numen node and runtime. github.com/numen-network/numen.
- [7] Ergo Platform. *Autolykos v2*, memory-hard PoW with lightweight verification.

Appendix A: Governance key set and protocol string

prime tables upgrade referenda. The validator stake-exemption list is separate on-chain storage managed by prime (Section 7.4); its live contents are queryable on any node. All keys below are verifiable on-chain; changes rotate through `prime.set_key` with a public event.

prime key:

```
nu7PrimeGGWHhqsFvKxLbmCsudaCDMDYaKwiDVR46k3AHUtYk
```

Airdrop genesis account:

```
nu5DropA1BwvKhPMDLrMXNFCmo4B9PK71iuZEMxESSJGvoyMg
```

PoScan protocol string:

```
poscan-v1
```

Core contributors

motoz Discord @numen_motoz Telegram @numen_motoz
fd Discord @fd3dpass Telegram @iayfd